

Instituto Superior Tecnológico Yavirac

Trabajo Colaborativo con Git y GitHub

Flujo de integración de código en equipos de desarrollo

Proyecto: *Calculadora Colaborativa*

Integrantes: Larissa Guaman, Andrew Carrera

Carrera: Desarrollo de Software

Curso: 5to “A”

Asignatura: DevOps (Development Operations)

Tema: Tarea 4.0 Trabajo Colaborativo en Parejas

Fecha: 5 de junio de 2026

Introducción

El presente documento recoge el trabajo colaborativo realizado por una pareja de estudiantes para comprender y aplicar el flujo de integración de código utilizado en equipos de desarrollo profesionales. El documento se divide en dos secciones principales: una investigación teórica de los conceptos fundamentales de control de versiones, Git, GitHub e integración continua; y una demostración práctica del proceso completo, evidenciada mediante capturas de pantalla. Durante la práctica, ambos integrantes intercambiaron los roles de creador/revisor y desarrollador colaborador, de modo que los dos realizaron todas las actividades requeridas.

1. Investigación

A continuación se presentan los conceptos fundamentales del control de versiones y la integración de código. Para cada uno se incluye su definición, utilidad, un ejemplo práctico y la fuente bibliográfica consultada.

1. Control de versiones

Definición	Es un sistema que registra los cambios realizados sobre uno o varios archivos a lo largo del tiempo, permitiendo recuperar versiones específicas más adelante. Lleva un historial completo de quién modificó qué, cuándo y por qué.
Utilidad	Permite trabajar en equipo sin sobrescribir el trabajo de otros, volver a versiones anteriores ante un error y mantener un registro ordenado de la evolución del proyecto.
Ejemplo práctico	Si un programador rompe una funcionalidad al editar el código, puede usar el control de versiones para regresar a la versión anterior que sí funcionaba, sin perder el resto de avances.
Fuente	Chacon, S. & Straub, B. (2014). Pro Git (2. ^a ed.). Apress. https://git-scm.com/book/es/v2

2. Git

Definición	Es un sistema de control de versiones distribuido y de código abierto, creado por Linus Torvalds en 2005. Cada desarrollador tiene una copia completa del historial del proyecto en su máquina.
Utilidad	Permite gestionar el historial de cambios de forma local y rápida, crear ramas para trabajar en paralelo y sincronizar el trabajo con otros desarrolladores de manera eficiente.
Ejemplo práctico	Con el comando 'git commit' un desarrollador guarda una versión de su trabajo, y con 'git push' la envía a un servidor remoto para compartirla con su equipo.
Fuente	Git. (s.f.). Documentación oficial de Git. Recuperado de https://git-scm.com/doc

3. GitHub

Definición	Es una plataforma web que aloja repositorios Git en la nube y añade herramientas de colaboración como Pull Requests, control de acceso, seguimiento de incidencias (issues) y automatización (Actions).
Utilidad	Facilita el trabajo colaborativo a distancia, sirve como respaldo remoto del código y permite revisar cambios antes de integrarlos, además de mostrar el trabajo de forma pública.
Ejemplo práctico	Un equipo sube su proyecto a un repositorio de GitHub; cada integrante propone cambios mediante Pull Requests que son revisados y aprobados antes de fusionarse.
Fuente	GitHub. (s.f.). GitHub Docs. Recuperado de https://docs.github.com

4. Repositorio local

Definición	Es la copia del proyecto que reside en la computadora del desarrollador. Contiene todos los archivos del proyecto junto con la carpeta oculta '.git', donde se almacena el historial completo de versiones.
Utilidad	Permite trabajar sin conexión a internet, realizar commits y crear ramas localmente, y experimentar con cambios antes de compartirlos con el equipo.
Ejemplo práctico	Al ejecutar 'git clone', se descarga una copia del proyecto a la máquina; esa copia, con su historial, es el repositorio local.
Fuente	Atlassian. (s.f.). ¿Qué es un repositorio de Git? Recuperado de https://www.atlassian.com/es/git/tutorials/what-is-version-control

5. Repositorio remoto

Definición	Es la versión del repositorio alojada en un servidor accesible por la red, como GitHub. Funciona como punto central donde los integrantes del equipo publican y obtienen los cambios.
Utilidad	Sirve como copia de respaldo y como punto de sincronización entre los repositorios locales de todos los colaboradores, permitiendo el trabajo en equipo.
Ejemplo práctico	Con 'git push origin main' un desarrollador envía sus commits al repositorio remoto, y con 'git pull' otro integrante los descarga a su máquina.
Fuente	GitHub. (s.f.). About remote repositories. Recuperado de https://docs.github.com/en/get-started/getting-started-with-git/about-remote-repositories

6. Commit

Definición	Es una instantánea (snapshot) del estado del proyecto en un momento determinado. Cada commit guarda los cambios realizados junto con un mensaje descriptivo, un autor y una fecha.
Utilidad	Permite registrar avances de forma ordenada, documentar qué se cambió en cada paso y poder regresar a cualquier punto del historial.
Ejemplo práctico	Tras agregar una función nueva, el desarrollador ejecuta: <code>git commit -m "Agrega función de resta a la calculadora"</code> , creando un punto en el historial.
Fuente	Git. (s.f.). git-commit Documentation. Recuperado de https://git-scm.com/docs/git-commit

7. Branch (rama)

Definición	Es una línea independiente de desarrollo dentro del repositorio. Permite trabajar en una funcionalidad o corrección aislada sin afectar la rama principal (main).
Utilidad	Facilita el desarrollo en paralelo: varios integrantes pueden trabajar en distintas funcionalidades al mismo tiempo y luego integrar sus cambios de forma controlada.
Ejemplo práctico	Con 'git checkout -b funcionalidad-resta' se crea y cambia a una rama nueva para desarrollar la resta, sin modificar la rama main hasta que esté lista.
Fuente	Atlassian. (s.f.). Git Branch. Recuperado de https://www.atlassian.com/es/git/tutorials/using-branches

8. Merge

Definición	Es el proceso de combinar los cambios de una rama dentro de otra, integrando el trabajo realizado por separado en una sola línea de desarrollo.
Utilidad	Permite unificar el trabajo de distintas ramas o colaboradores en la rama principal, consolidando las funcionalidades terminadas.
Ejemplo práctico	Tras aprobar un Pull Request, el responsable hace 'Merge pull request' para integrar la rama 'funcionalidad-resta' a la rama 'main'.
Fuente	Git. (s.f.). git-merge Documentation. Recuperado de https://git-scm.com/docs/git-merge

9. Pull Request

Definición	Es una solicitud para integrar los cambios de una rama en otra dentro de plataformas como GitHub. Abre un espacio donde el equipo puede revisar, comentar y aprobar el código antes de fusionarlo.
Utilidad	Promueve la revisión de código entre pares, mejora la calidad del software y deja constancia de quién revisó y aprobó cada cambio.
Ejemplo práctico	Un colaborador sube su rama y crea un Pull Request titulado "Agrega función de multiplicación"; otro integrante lo revisa, lo aprueba y lo fusiona.
Fuente	GitHub. (s.f.). About pull requests. Recuperado de https://docs.github.com/en/pull-requests

10. Fork

Definición	Es una copia personal y completa de un repositorio de otro usuario en la propia cuenta de GitHub. Permite trabajar libremente sobre el proyecto sin afectar el repositorio original.
Utilidad	Es útil para contribuir a proyectos de código abierto: se trabaja sobre la copia (fork) y luego se proponen los cambios al proyecto original mediante un Pull Request.
Ejemplo práctico	Un desarrollador hace 'Fork' de un proyecto público, corrige un error en su copia y envía un Pull Request al repositorio original para que el cambio sea considerado.
Fuente	GitHub. (s.f.). Fork a repository. Recuperado de https://docs.github.com/en/get-started/quickstart/fork-a-repo

11. Conflictos de integración

Definición	Ocurren cuando dos ramas modifican la misma parte de un mismo archivo de forma distinta y Git no puede decidir automáticamente cuál versión conservar al fusionar.
Utilidad	Obligan a una revisión manual que garantiza que ningún cambio se pierda; resolverlos correctamente mantiene la integridad del código integrado.
Ejemplo práctico	Dos integrantes editan la misma línea del título de la calculadora en ramas separadas; al fusionar la segunda, Git marca el conflicto con <<<<<<<, ===== y >>>>>>> para que un humano lo resuelva.
Fuente	Atlassian. (s.f.). Merge conflicts. Recuperado de https://www.atlassian.com/es/git/tutorials/using-branches/merge-conflicts

12. Integración Continua (CI)

Definición	Es una práctica de desarrollo en la que los integrantes fusionan su código en un repositorio compartido con frecuencia, ejecutando de forma automática compilaciones y pruebas en cada cambio.
Utilidad	Detecta errores de manera temprana, mantiene el código siempre en un estado funcional y reduce los problemas al integrar el trabajo de varias personas.
Ejemplo práctico	Cada vez que se abre un Pull Request, GitHub Actions ejecuta automáticamente las pruebas con pytest; si fallan, el equipo se entera de inmediato.
Fuente	Atlassian. (s.f.). ¿Qué es la integración continua? Recuperado de https://www.atlassian.com/es/continuous-delivery/continuous-integration

13. Entrega Continua (Continuous Delivery)

Definición	Es una práctica que extiende la integración continua asegurando que el software pueda ser liberado a producción en cualquier momento, dejando el despliegue final como una decisión manual.
Utilidad	Permite tener siempre una versión lista para publicar, reduciendo el riesgo y el tiempo de las entregas al automatizar las etapas previas al despliegue.
Ejemplo práctico	Tras pasar todas las pruebas automáticas, una nueva versión de la aplicación queda preparada y validada; basta con que un responsable apruebe su publicación.
Fuente	Atlassian. (s.f.). Continuous delivery. Recuperado de https://www.atlassian.com/es/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment

14. Despliegue Continuo (Continuous Deployment)

Definición	Es la práctica en la que cada cambio que supera las pruebas automáticas se publica automáticamente en producción, sin intervención manual.
Utilidad	Acelera la llegada de nuevas funciones a los usuarios y elimina pasos manuales, siempre que exista una sólida cobertura de pruebas automatizadas.
Ejemplo práctico	Un equipo configura su flujo para que, al aprobar y fusionar un cambio que pasa todas las pruebas, este se publique solo en el sitio web de producción.
Fuente	Atlassian. (s.f.). Continuous deployment. Recuperado de https://www.atlassian.com/es/continuous-delivery/continuous-deployment

15. GitHub Actions

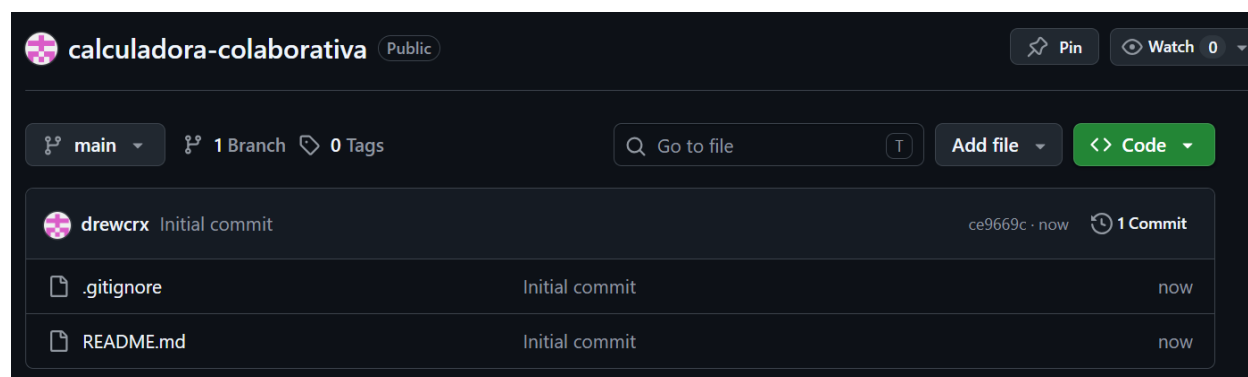
Definición	Es la herramienta de automatización integrada en GitHub que permite crear flujos de trabajo (workflows) que se ejecutan automáticamente ante eventos como un push o un Pull Request.
Utilidad	Permite implementar integración y entrega continua directamente en el repositorio: ejecutar pruebas, compilar el proyecto o desplegarlo de forma automatizada.
Ejemplo práctico	Un archivo <code>.github/workflows/ci.yml</code> indica que, en cada push o Pull Request a main, se instale Python y se ejecuten las pruebas con pytest, mostrando un check verde si todo pasa.
Fuente	GitHub. (s.f.). GitHub Actions documentation. Recuperado de https://docs.github.com/en/actions

2. Demostración Práctica

En esta sección se documenta, mediante capturas de pantalla y explicaciones breves, el desarrollo de la práctica realizada por la pareja. Cada captura se acompaña de una explicación que indica qué actividad se realizó y cuál fue el resultado obtenido.

1. Creación del repositorio en GitHub

Actividad realizada: Se evidencia la creación del repositorio en GitHub, donde se observa la rama principal, los archivos iniciales y el primer commit realizado.



Resultado obtenido: El repositorio quedó creado correctamente y listo para iniciar el trabajo colaborativo mediante ramas, commits, Pull Requests y merges.

2. Creación de ramas de trabajo por cada integrante

Actividad realizada: Se evidencia la sección de ramas activas del repositorio calculadora-colaborativa en GitHub. En la captura se observan varias ramas de trabajo creadas para el desarrollo colaborativo, como configuracion-ci, mejora-encabezado, funcionalidad-multiplicacion y funcionalidad-resta. También se visualiza que cada rama está relacionada con su respectivo Pull Request, lo que demuestra el uso de ramas independientes para integrar cambios de forma ordenada.

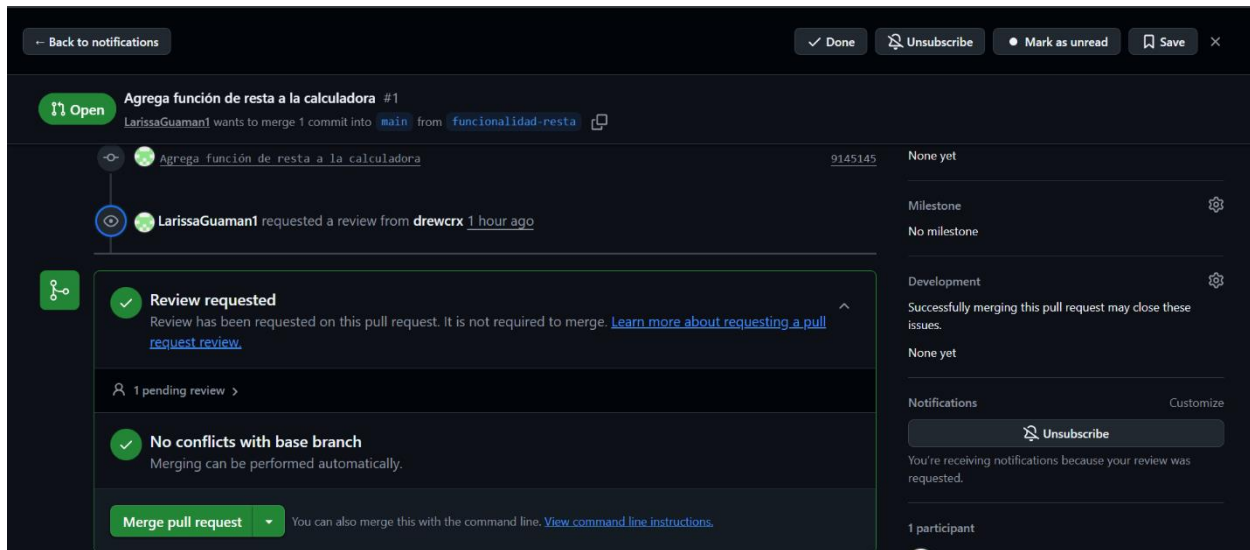
Branch	Updated	Check status	Behind	Ahead	Pull request
configuracion-ci	27 minutes ago	✓ 1 / 1	1	0	#5
mejora-encabezado	35 minutes ago		3	0	#4
funcionalidad-multiplicacion	12 hours ago		8	0	#2
funcionalidad-resta	13 hours ago		10	0	#1

Resultado obtenido: El repositorio cuenta con varias ramas activas utilizadas para trabajar funcionalidades separadas. Esto demuestra una correcta organización del trabajo colaborativo, permitiendo que los cambios sean revisados mediante Pull Requests antes de integrarse a la rama principal.

3. Commits realizados por ambos integrantes

Actividad realizada: Se evidencia el historial de commits realizados en el repositorio calculadora-colaborativa. En la captura se observan los cambios desarrollados en diferentes momentos, como el commit inicial, la creación de la estructura base de la calculadora con la función suma, la incorporación de la función resta, la función multiplicación y los merges realizados mediante Pull Requests.

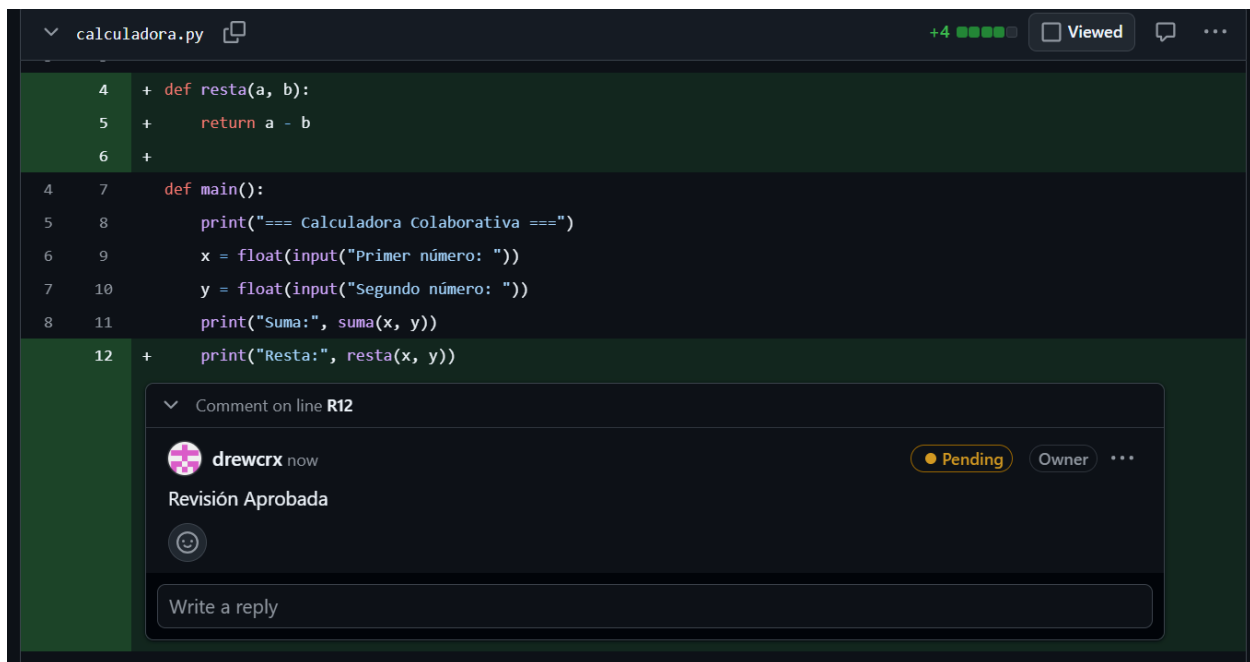
Merge pull request #2 from drewcrx/funcionalidad-multiplicacion	Verified	ea2b364
LarissaGuaman1 authored 12 hours ago		
Agrega función de multiplicación a la calculadora		67d5de3
drewcrx committed 12 hours ago		
Merge pull request #1 from drewcrx/funcionalidad-resta	Verified	c5109f9
drewcrx authored 12 hours ago		
Agrega función de resta a la calculadora		9145145
LarissaGuaman1 committed yesterday		
Agrega estructura base de la calculadora con función suma		e9d9584
drewcrx committed 2 days ago		
Initial commit	Verified	ce9669c
drewcrx authored 2 days ago		
Merge pull request #5 from drewcrx/configuracion-ci	Verified	59872f3
drewcrx authored 28 minutes ago - ✓ 1 / 1		
Agrega pruebas automaticas y workflow de GitHub Actions (CI)		925e3a0
drewcrx committed 30 minutes ago - ✓ 1 / 1		
Merge pull request #4 from drewcrx/mejora-encabezado	Verified	85b577f
drewcrx authored 36 minutes ago		
Resuelve conflicto de integración en el título de la calculadora		06a012e
drewcrx committed 38 minutes ago		
Merge pull request #3 from drewcrx/funcionalidad-division	Verified	46d3990
drewcrx authored 48 minutes ago		
Mejora el encabezado de la calculadora		0a5fe66
drewcrx committed 49 minutes ago		
Agrega función de división y actualiza título a version 1.0		c1c086a
LarissaGuaman1 committed 52 minutes ago		

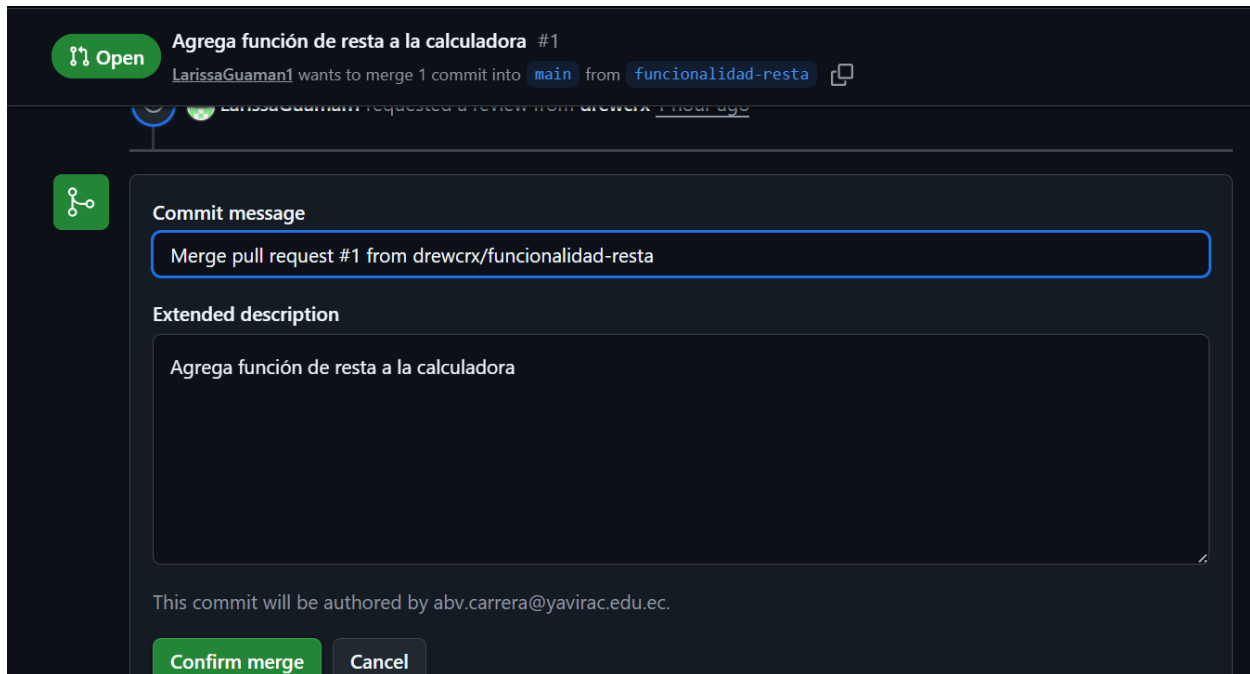


Resultado obtenido: El Pull Request fue creado correctamente y quedó disponible para revisión antes de realizar el merge. Esto demuestra el uso adecuado del flujo colaborativo de GitHub para integrar cambios de manera controlada.

6. Revisión de código

Actividad realizada: Se evidencia la revisión del archivo `calculadora.py` dentro del Pull Request en GitHub. En la captura se observan las líneas de código agregadas para la función de resta y su uso dentro del programa, junto con un comentario en línea realizado durante la revisión, indicando “Revisión Aprobada”.

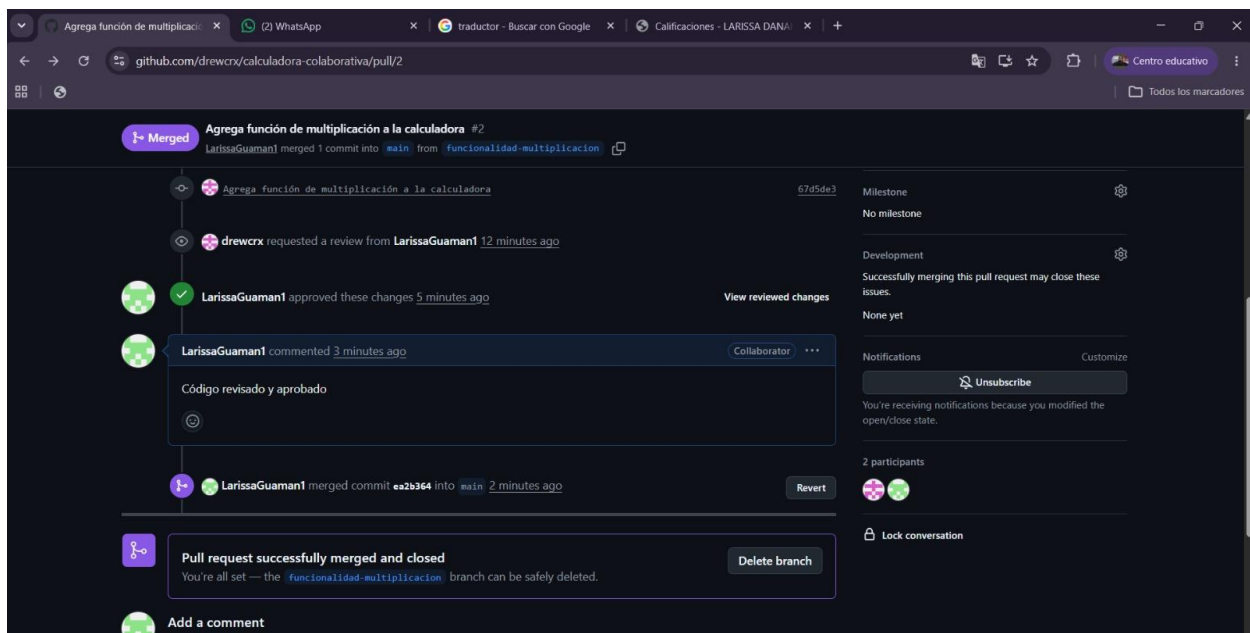




Resultado obtenido: La captura demuestra que el código fue revisado directamente sobre los cambios realizados antes de su integración. Esto permitió validar que la funcionalidad agregada era correcta y que el Pull Request podía continuar con el proceso de aprobación y merge.

7. Aprobación de Pull Requests

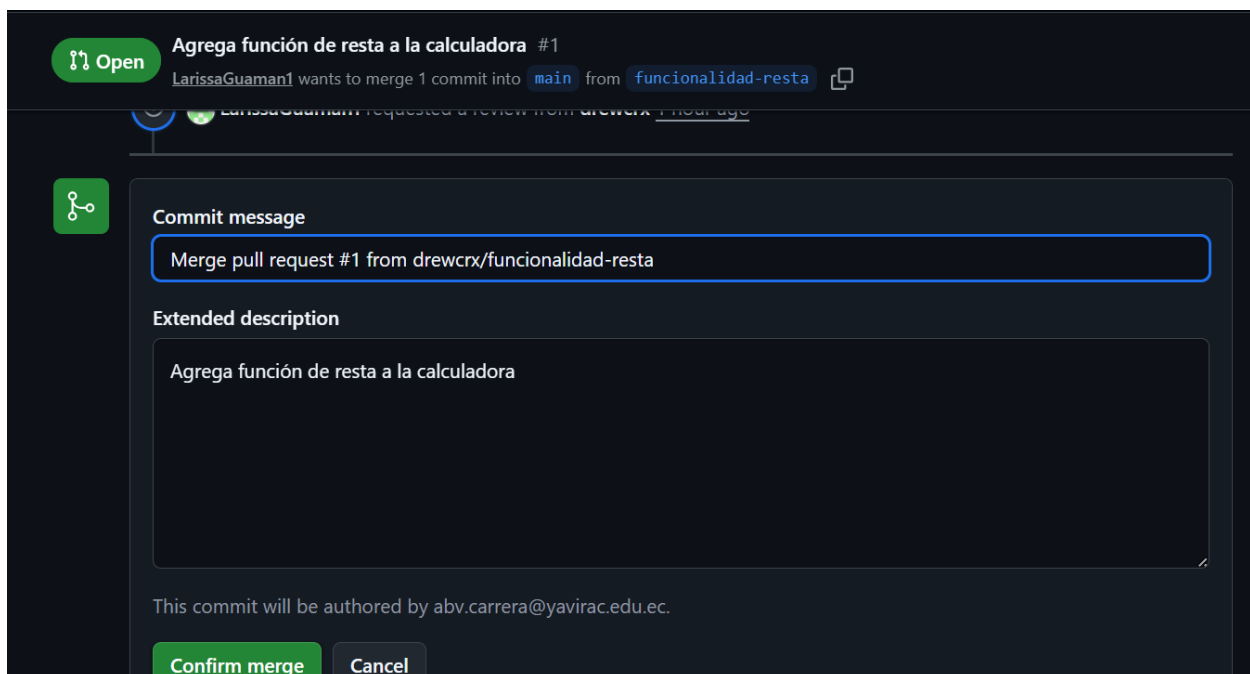
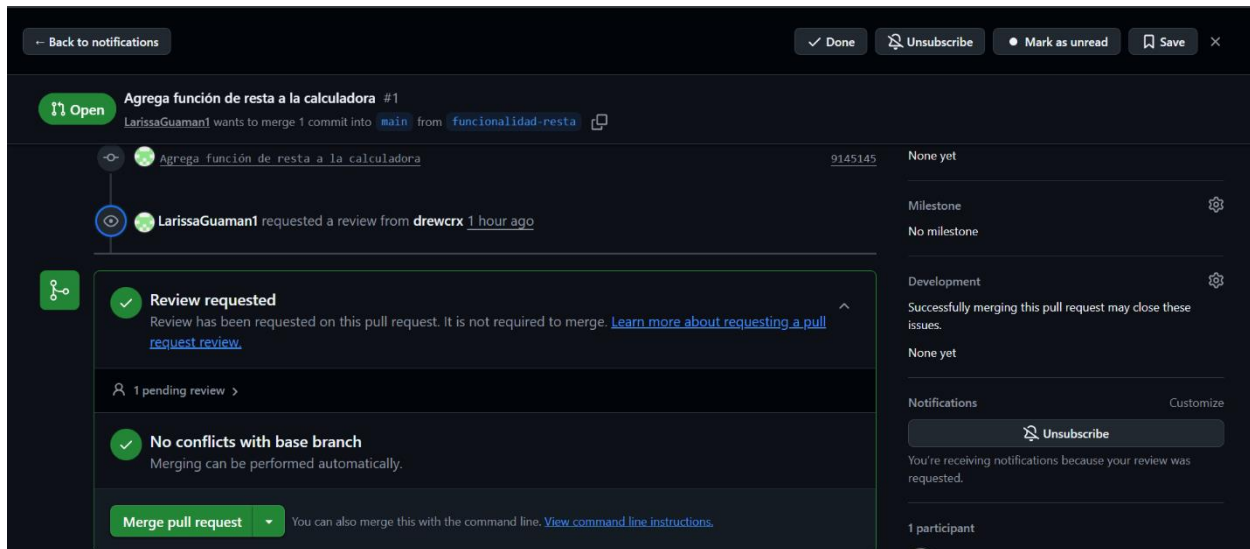
Actividad realizada: Se evidencia la aprobación del Pull Request #2 correspondiente a la funcionalidad de multiplicación de la calculadora. En la captura se observa que el colaborador LarissaGuaman1 revisó los cambios solicitados y aprobó el Pull Request, dejando además el comentario “Código revisado y aprobado”.



Resultado obtenido: El Pull Request fue aprobado correctamente después de la revisión de código, permitiendo continuar con el proceso de integración hacia la rama principal main. Esta evidencia demuestra el cumplimiento del flujo colaborativo de revisión y aprobación antes del merge.

8. Ejecución del proceso de Merge

Actividad realizada: Se evidencia el proceso de merge del Pull Request #1 correspondiente a la funcionalidad de resta. En la primera captura se observa que el Pull Request está listo para fusionarse con la rama principal main, mostrando el botón “Merge pull request”. En la segunda captura se muestra la pantalla de confirmación del merge, donde se genera el mensaje del commit de integración.



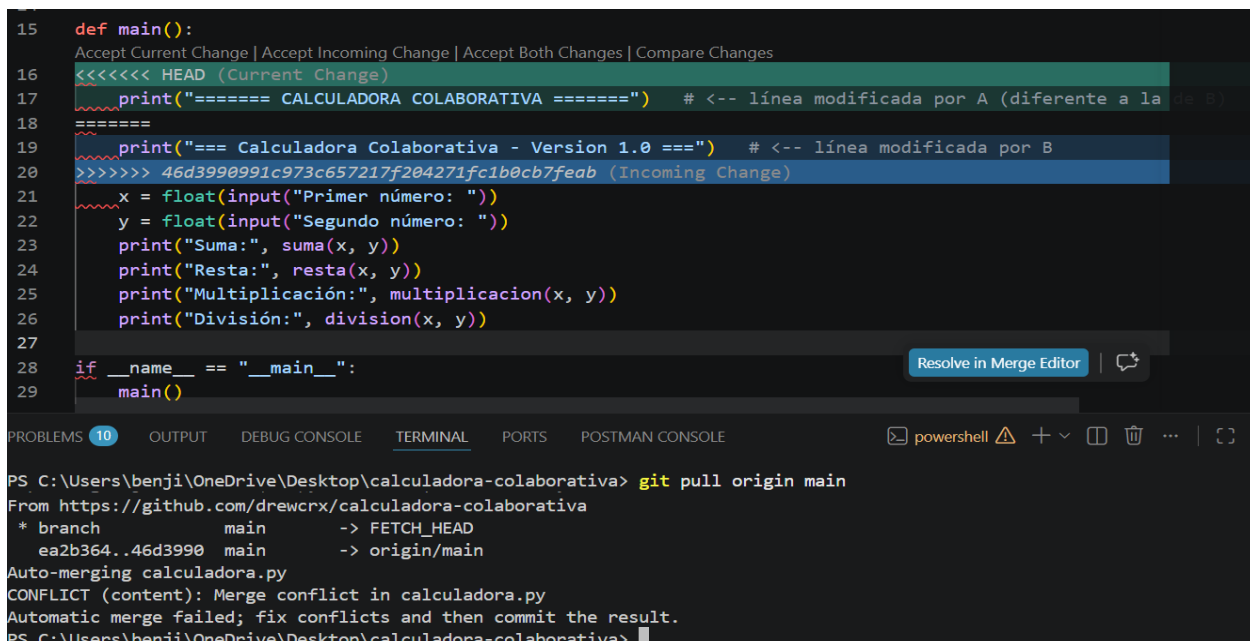
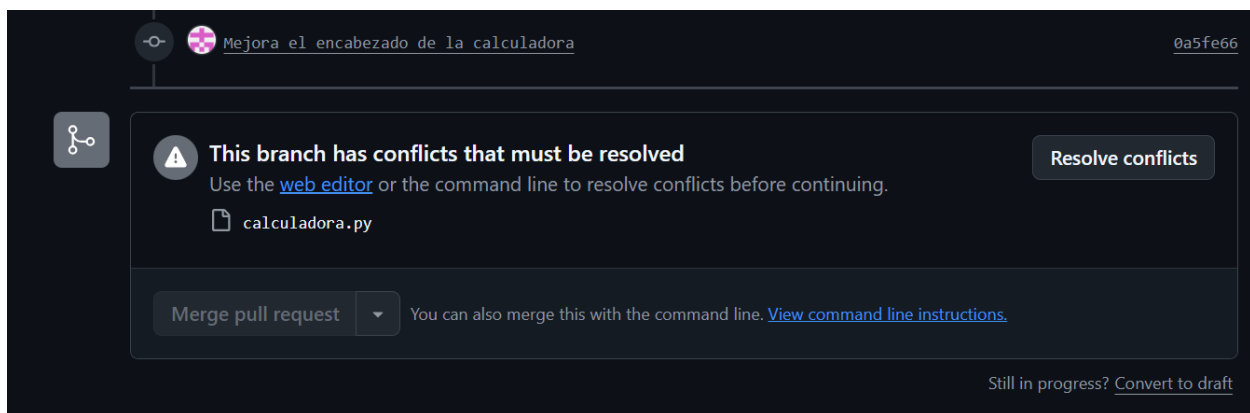
Resultado obtenido: El proceso de merge fue ejecutado correctamente desde GitHub, permitiendo integrar los cambios de la rama funcionalidad-resta hacia la rama principal main. Esto demuestra que el proyecto siguió el flujo colaborativo de revisión, aprobación e integración de cambios.

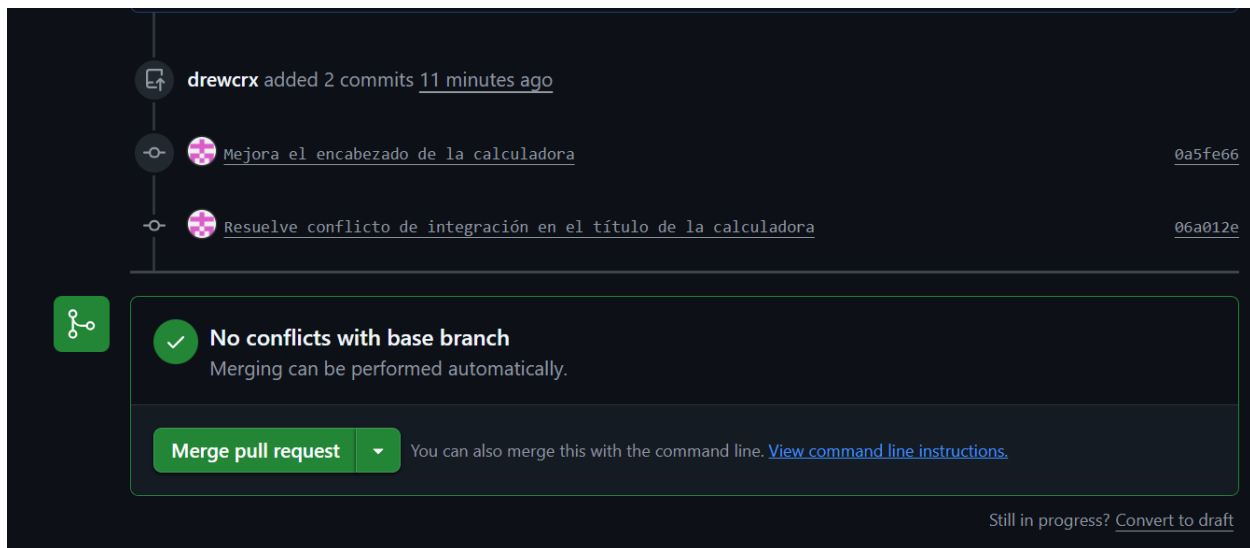
9. Generación y resolución de conflictos de integración

Actividad realizada: Se evidencia la generación de un conflicto de integración en el archivo `calculadora.py`. El conflicto se produjo porque dos ramas modificaron la misma línea del encabezado de la calculadora. En GitHub se observa el mensaje “This branch has conflicts that must be resolved”, indicando que el Pull Request no podía fusionarse automáticamente.

En la segunda captura se muestra el conflicto dentro de Visual Studio Code, donde aparecen los marcadores generados por Git: `<<<<<<<`, `=====` y `>>>>>>>`. También se observa el mensaje en la terminal indicando que el archivo `calculadora.py` tenía un conflicto de contenido durante el proceso de integración.

Finalmente, en la tercera captura se evidencia que el conflicto fue resuelto correctamente mediante un commit llamado “Resuelve conflicto de integración en el título de la calculadora”. GitHub muestra el mensaje “No conflicts with base branch”, lo que indica que el Pull Request quedó listo para realizar el merge.



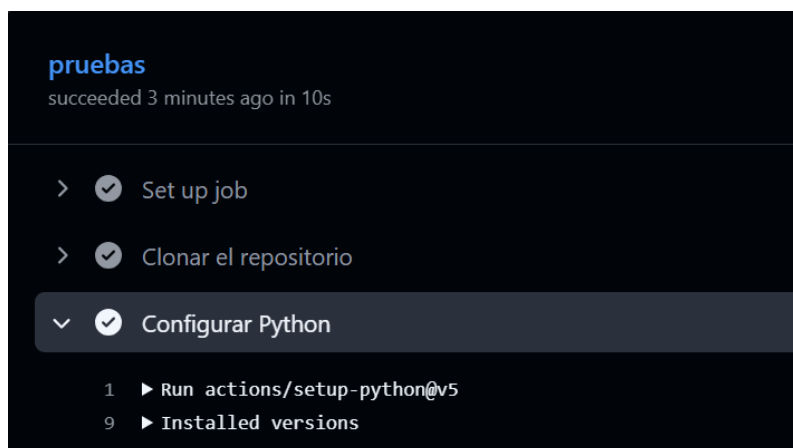
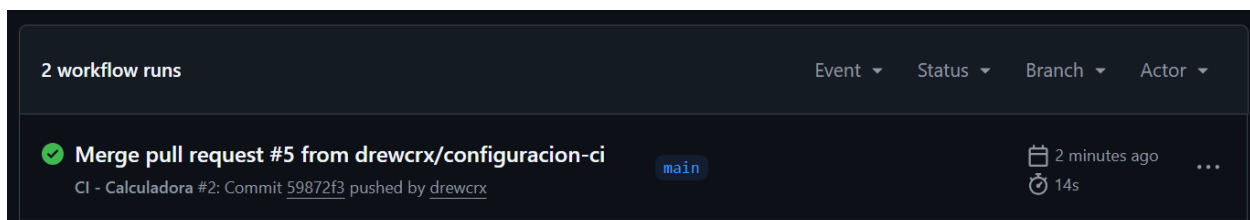


Resultado obtenido: El conflicto de integración fue identificado, corregido y resuelto correctamente. Se eliminó la duplicidad generada por los cambios en el encabezado de la calculadora y se dejó una única versión final del título. Como resultado, el Pull Request quedó sin conflictos y disponible para ser integrado a la rama principal main.

10. Ejecución exitosa del workflow de GitHub Actions

Actividad realizada: Se evidencia la ejecución del workflow de GitHub Actions configurado para el proyecto calculadora-colaborativa. En la captura se observa que el workflow se ejecutó después del merge del Pull Request #5 desde la rama configuracion-ci hacia main, mostrando un estado exitoso con el check verde.

Además, se agregan evidencias del proceso interno del workflow, donde se muestra la configuración de Python, la instalación de dependencias y la ejecución de las pruebas automáticas.



```

✓ Instalar dependencias

1 ▶ Run pip install pytest
11 Collecting pytest
12   Downloading pytest-9.0.3-py3-none-any.whl.metadata (7.6 kB)
13 Collecting iniconfig>=1.0.1 (from pytest)
14   Downloading iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
15 Collecting packaging>=22 (from pytest)
16   Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)
17 Collecting pluggy<2,>=1.5 (from pytest)
18   Downloading pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
19 Collecting pygments>=2.7.2 (from pytest)
20   Downloading pygments-2.20.0-py3-none-any.whl.metadata (2.5 kB)
21 Downloading pytest-9.0.3-py3-none-any.whl (375 kB)
22 Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
23 Downloading iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
24 Downloading packaging-26.2-py3-none-any.whl (100 kB)
25 Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
26   _____ 1.2/1.2 MB 21.1 MB/s 0:00:00
27 Installing collected packages: pygments, pluggy, packaging, iniconfig, pytest
28
29 Successfully installed iniconfig-2.3.0 packaging-26.2 pluggy-1.6.0 pygments-2.20.0 pytest-9.0.3
30
31 Notice: A new release of pip is available: 26.1.1 -> 26.1.2
32 Notice: To update, run: pip install --upgrade pip

✓ Ejecutar pruebas 0s

1 ▶ Run pytest -v
11 ===== test session starts =====
12 platform linux -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0 -- /opt/hostedtoolcache/Python/3.11.15/x64/bin/python
13 cachedir: .pytest_cache
14 rootdir: /home/runner/work/calculadora-colaborativa/calculadora-colaborativa
15 collecting ... collected 4 items
16
17 test_calculadora.py::test_suma PASSED [ 25%]
18 test_calculadora.py::test_resta PASSED [ 50%]
19 test_calculadora.py::test_multiplicacion PASSED [ 75%]
20 test_calculadora.py::test_division PASSED [100%]
21
22 ===== 4 passed in 0.01s =====

```

Resultado obtenido: El workflow de integración continua se ejecutó correctamente, validando que el proyecto puede ser probado automáticamente desde GitHub Actions. Esto demuestra que la configuración CI funciona y que los cambios integrados en la rama principal pasan correctamente por el proceso de verificación automática.

11. Estado final del repositorio después de integrar todos los cambios

Actividad realizada: Se evidencia el estado final del repositorio calculadora-colaborativa en la rama principal main después de integrar todos los cambios realizados durante la actividad. En la captura se observa el último merge correspondiente al Pull Request #5 desde la rama configuracion-ci, además de los archivos finales del proyecto: .github/workflows, .gitignore, README.md, calculadora.py y test_calculadora.py.

También se visualiza el historial de 13 commits, lo que demuestra el avance progresivo del proyecto mediante ramas, commits, Pull Requests, resolución de conflictos y configuración de integración continua.


calculadora-colaborativa
Public
Pin
Watch 0

main
5 Branches
0 Tags

Add file
Code


drewcrx Merge pull request #5 from drewcrx/configuracion-ci
 59872f3 · 4 minutes ago
13 Commits

 .github/workflows	Agrega pruebas automaticas y workflow de GitHub Actions ...	6 minutes ago
 .gitignore	Initial commit	2 days ago
 README.md	Initial commit	2 days ago
 calculadora.py	Resuelve conflicto de integración en el título de la calculado...	15 minutes ago
 test_calculadora.py	Agrega pruebas automaticas y workflow de GitHub Actions ...	6 minutes ago

Resultado obtenido: El repositorio quedó actualizado correctamente en la rama principal main, integrando todas las funcionalidades desarrolladas, las pruebas automáticas y el workflow de GitHub Actions. Esto demuestra que el trabajo colaborativo fue completado exitosamente y que el proyecto quedó en un estado final funcional y organizado.

3. Información Adicional

Datos de los integrantes

Nombre completo (Estudiante A): Andrew Carrera

Nombre completo (Estudiante B): Larissa Guaman

Carrera y paralelo: Desarrollo de Software 5to “A”

URL del repositorio GitHub: <https://github.com/drewcrx/calculadora-colaborativa.git>

Conclusiones individuales

Conclusión del Estudiante A:

Durante el desarrollo de esta actividad pude comprender mejor el uso de Git y GitHub dentro de un trabajo colaborativo.

Aprendí a crear ramas, realizar cambios en el código, hacer commits y publicar los avances en el repositorio remoto mediante push.

También participé en la creación y revisión de Pull Requests, lo que permitió integrar los cambios de forma ordenada a la rama principal.

La resolución de conflictos me ayudó a entender la importancia de revisar cuidadosamente el código antes de hacer un merge.

Finalmente, el uso de GitHub Actions permitió comprobar que el proyecto puede validarse automáticamente, mejorando la calidad del trabajo realizado.

Conclusión del Estudiante B:

Esta práctica me permitió reforzar mis conocimientos sobre el flujo colaborativo de desarrollo usando Git y GitHub.

Pude trabajar en ramas independientes para agregar funcionalidades a la calculadora sin afectar directamente la rama principal.

También comprendí la importancia de los Pull Requests, ya que permiten revisar, aprobar y controlar los cambios antes de integrarlos.

La generación y resolución de conflictos fue una experiencia importante, porque demuestra una situación real que puede ocurrir en proyectos de software.

Como resultado, se logró completar un proyecto organizado, con commits, merges, pruebas automáticas y un workflow de integración continua funcionando correctamente.

Bibliografía

- Atlassian. (s. f.-a). *Integración y entrega continua*. <https://www.atlassian.com/es/continuous-delivery>
- Atlassian. (s. f.-b). *Tutoriales de Git*. <https://www.atlassian.com/es/git/tutorials>
- Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. <https://git-scm.com/book/es/v2>
- Git. (s. f.). *Documentación oficial de Git*. <https://git-scm.com/doc>
- GitHub. (s. f.-a). *GitHub Actions documentation*. <https://docs.github.com/en/actions>
- GitHub. (s. f.-b). *GitHub Docs*. <https://docs.github.com>